

CineSeat

System Architecture Design and Module Breakdown

Prepared by: Nunna Ravi Varma

Date: August 12, 2026

1. System Overview

Cine Seat is a real-time seat reservation platform for movie theatres. It is modeled on the booking flows of Book My Show and Ticketmaster, and must support many theatres, multiple screens per theatre, and multiple show times per screen, all while allowing hundreds of concurrent users to compete for the same limited pool of seats.

At its core, Cine Seat solves a resource-contention problem that is structurally identical to a parking-management system: a fixed number of discrete slots (seats) must be shown as available in real time, temporarily held for a single user during checkout, automatically released if payment is not completed, and finally committed once payment succeeds. The Week 1 Software Requirements Document already established the functional scope, non-functional requirements, and the concurrency/locking strategy; this document builds on that foundation to define the system's architecture design and module breakdown.

The architecture is organized around four goals:

- **Correctness under concurrency** — no two users can ever be confirmed for the same seat, even under simultaneous requests.
- **Real-time responsiveness** — seat availability shown to a user must reflect the true current state within milliseconds.
- **Modularity** — each capability (catalog, inventory, reservation, payment, notification) is independently developable, testable, and deployable.
- **Scalability** — the system must tolerate traffic spikes around popular show releases without seat-level contention becoming a bottleneck.

2. High-Level Architecture

Cine Seat follows a layered, modular-monolith-ready architecture (deployable first as a set of tightly-coordinated Spring Boot modules, and horizontally splittable into micro services as load grows). Client applications never talk to the database or cache directly; every request passes through an API Gateway before reaching the appropriate service module. Figure 1 shows the complete layered view, from client applications down to external integrations.

Cine Seat- High-Level System Architecture(Layered View)

CLIENT LAYER

Web App (React)	Mobile App (Android/IOS)	Admin Console (Theatre/Show management)
------------------------	---------------------------------	---

API GATEWAY LAYER

API Gateway (Spring Cloud Gateway – Routing, Auth Filter, Rate Limiting)

APPLICATION SERVICE LAYER

User & Auth Service	Catalog Service (Theatres/Shows)	Seat Inventory & Looking Service	Booking & Reservation Service	Payment Service	Notification Service
--------------------------------	--	---	--	------------------------	-----------------------------

CONCURRENCY/CACHING LAYER

Redis Cache (Seat Status Hash Map)	Redisson (Distributed locks + Hold-Timer scheduler)
---	--

DATA LAYER

PostgreSQL (Bookings,Shows,Users,Seats)	Message Queue (Kafka/RabbitMQ-Async Events)
--	--

EXTERNAL INTEGRATIONS

Payment Gateway (Razorpay/Stripe)	SMS/Email/Push (Twilio,FCM,SES)	Theater Partner APIs
---	---	-----------------------------

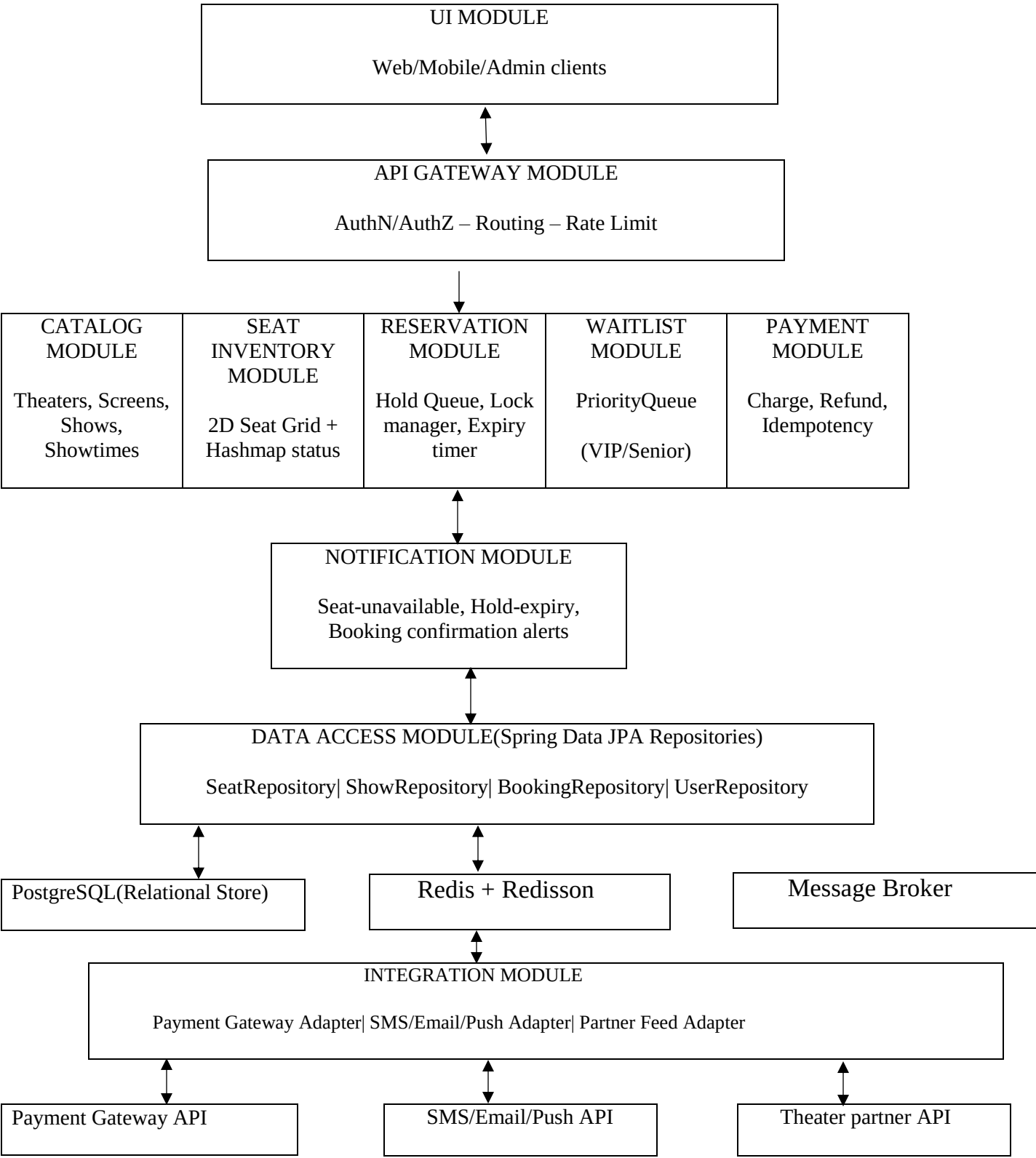
Layer responsibilities:

- Client Layer — Web app, mobile app, and an admin console for theatre/show management, all consuming the same API contract.
- API Gateway Layer — Single entry point handling authentication, request routing to the correct service, and rate limiting to protect the Seat Inventory & Locking Service during high-demand releases.
- Application Service Layer — Independently deployable Spring Boot services, each owning one bounded context (users, catalog, seat inventory, booking, payment, notification).
- Concurrency / Caching Layer — Redis holds the live seat-status HashMap for O(1) lookups; Redisson provides distributed locks and the scheduler that drives 10-minute hold expiry, so correctness under concurrency does not depend on any single service instance.
- Data Layer — PostgreSQL is the system of record for bookings, shows, users, and seats; a message queue (Kafka or RabbitMQ) carries asynchronous events between services (e.g., booking-confirmed → notification).
- External Integrations — Payment gateway (Razorpay/Stripe), messaging providers (Twilio, FCM, SES) for SMS/email/push, and theatre partner feeds for show data sync.

3. Module Breakdown

The system is decomposed into ten modules spanning user interface, business logic, data access, and integration concerns. Figure 2 shows how these modules connect; Table 1 details each module's responsibilities, inputs, outputs, and communication pattern.

CINESEAT- MODULE BREAKDOWN & INTERACTION



Module	Responsibilities	Inputs	Outputs	Inter-Module Communication
UI Module	Renders seat maps, show listings, checkout flow, and admin screens across web, mobile, and admin console.	User interactions (seat taps, search, filters)	REST/WebSocket calls to API Gateway	Talks only to the API Gateway; receives real-time seat updates via WebSocket/SSE.
API Gateway Module	AuthN/AuthZ, request routing, rate limiting, request/response shaping.	HTTP requests from clients	Routed requests to internal services	Fronts every downstream module; no module is publicly reachable except through it.
Catalog Module	Manages theatres, screens, shows, and showtimes; exposes searchable show listings.	Admin CRUD operations, partner feed syncs	Show/showtime data	Read by Reservation and Seat Inventory modules when a user selects a showtime.
Seat Inventory & Locking Module	Maintains the 2D seat-grid layout per screen and the live seat-status HashMap; exposes O(1) seat-status lookups.	Seat selection requests, lock/unlock events	Current seat status (AVAILABLE / HELD / BOOKED)	Consulted by Reservation Module before every hold; updated by Redisson lock callbacks.
Reservation Module	Orchestrates the hold workflow: acquires a distributed lock, places the seat on the hold queue, and starts the auto-expiry timer.	Seat hold requests	Hold confirmation, expiry/release events	Coordinates with Seat Inventory, Payment, and Notification modules; publishes events to the message broker.
Waitlist Module	Maintains a PriorityQueue of waiting users per sold-out show, prioritizing VIP/senior-citizen bookings.	Waitlist join requests, seat-released events	Waitlist position, claim offers	Subscribes to seat-released events from the Reservation Module.

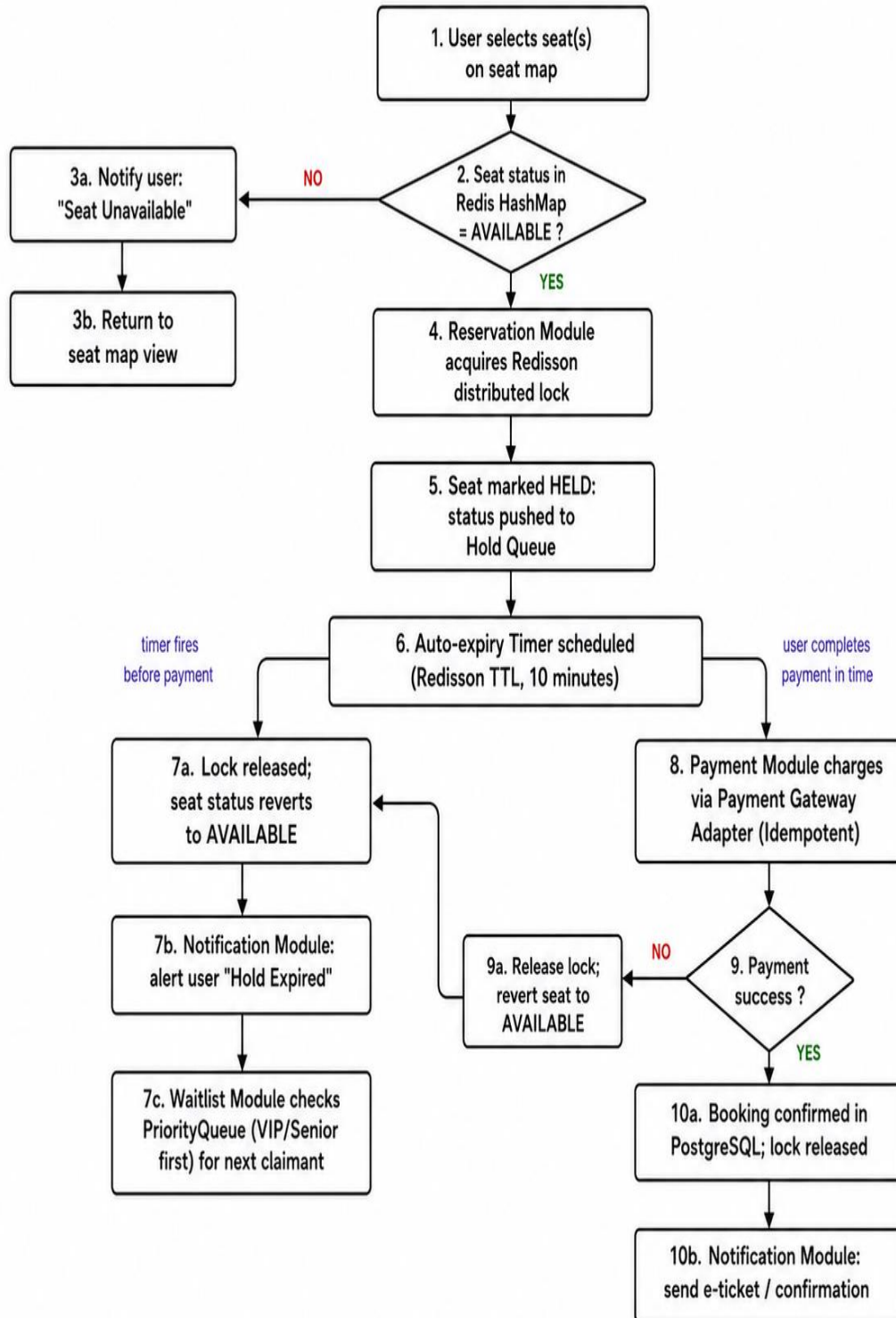
Module	Responsibilities	Inputs	Outputs	Inter-Module Communication
Payment Module	Initiates and confirms payment, guarantees idempotency so a retried request cannot double-charge.	Payment initiation request, gateway callback	Payment success/failure event	Calls the Integration Module's payment adapter; notifies Reservation Module of the outcome.
Notification Module	Sends seat-unavailable, hold-expiry, and booking-confirmation alerts to users.	Domain events (hold expired, payment confirmed, seat taken)	SMS / email / push notifications	Triggered asynchronously by Reservation and Payment module events via the message broker.
Data Access Module	Encapsulates all persistence logic through Spring Data JPA repositories.	Domain objects from service modules	Persisted/retrieved entities	Sole path from any service module to PostgreSQL; enforces pessimistic/optimistic locking rules at the query level.
Integration Module	Adapts external APIs (payment gateway, messaging providers, theatre partner feeds) behind a stable internal interface.	Outbound requests from Payment and Notification modules	External API responses	Only module permitted to make outbound calls to third-party systems.

Table 1. Detailed module breakdown.

4. Seat Reservation & Hold-Expiry Flow

The reservation flow is the heart of CineSeat's concurrency design and directly implements the locking strategy defined in Section 3.4 of the SRD. Figure 3 traces a single seat-hold request from selection through either payment confirmation or automatic release.

CineSeat – Seat Reservation & Hold-Expiry Flow



Key mechanics illustrated in the flow:

- Availability check — the Reservation Module first reads seat status from the Redis-backed HashMap, giving an $O(1)$ check before any lock is attempted.
- Distributed locking — Redisson acquires a lock on the specific seat key so that only one request can transition a seat from AVAILABLE to HELD, even across multiple service instances; this is the distributed-system analogue of the in-process ReentrantLock used during single-node testing.
- Hold queue — held seats are tracked in a queue structure so hold state, position, and expiry can be inspected and swept efficiently.
- Auto-expiry timer — a 10-minute TTL is attached to the lock itself (Redisson's scheduler), so expiry does not depend on any application thread staying alive; if payment is not completed in time, the lock's TTL expiry triggers seat release automatically.
- Waitlist hand-off — on release (whether by expiry or explicit cancellation), the Waitlist Module's PriorityQueue is checked so VIP/senior-citizen claimants are offered the seat before general users.
- Idempotent payment — the Payment Module uses an idempotency key per hold so that network retries during checkout cannot result in a duplicate charge.

5. Technology Stack and Rationale

The stack was finalized after evaluating alternatives against CineSeat's core requirement: correctness under high-concurrency seat contention, with room to scale horizontally.

Module / Concern	Technology	Rationale
Backend framework	Java 17 + Spring Boot	Spring Boot's mature ecosystem (Spring Data, Spring Security, Spring Cloud Gateway) covers nearly every layer of the architecture with production-grade, well-documented modules, reducing integration risk for a solo-built system.
Persistence	Spring Data JPA + PostgreSQL	PostgreSQL provides ACID transactions and row-level locking (SELECT ... FOR UPDATE), which underpins the pessimistic-locking fallback path for seat confirmation; Spring Data JPA keeps repository code declarative and testable.
Caching & real-time state	Redis	In-memory HashMap-style storage gives O(1) seat-status reads/writes, which is essential for showing live availability to many concurrent users without hitting PostgreSQL on every seat-map render.
Distributed locking & timers	Redisson	Redisson's RLock with a built-in leaseTime implements the 10-minute auto-expiring hold without a separate scheduler process, and works correctly across multiple service instances — unlike a plain in-JVM ReentrantLock.
Async messaging	Kafka / RabbitMQ	Decouples the Reservation and Payment modules from the Notification module, so a slow SMS/email provider cannot block the booking critical path.
API layer	Spring Cloud Gateway	Centralizes authentication and rate limiting in one place, protecting the Seat Inventory & Locking Service from being overwhelmed during a popular show's release.
Testing	JUnit 5 + Mockito	Enables isolated unit tests of locking logic (e.g., simulating concurrent hold attempts) without standing up the full Redis/PostgreSQL stack for every test run.
Client applications	React (web), native/cross-platform mobile	React's component model maps naturally onto a 2D seat-grid UI, and its ecosystem has mature WebSocket libraries for consuming real-time seat updates.

6. Scalability and Maintainability

6.1 Scalability

- Horizontal scaling of stateless services — every Application Service Layer module is stateless; seat lock state lives in Redis/Redisson rather than in-process, so any number of service instances can be added behind the API Gateway without breaking lock correctness.
- Read/write separation — seat-status reads are served from Redis; only lock acquisition and final booking commits touch PostgreSQL, keeping the database's write load proportional to actual bookings rather than every seat-map view.
- Event-driven decoupling — the message broker lets Notification and Waitlist processing scale independently of the booking critical path, and lets new consumers (e.g., analytics) be added without touching existing modules.
- Partitioning by theatre/show — seat-lock keys are namespaced by theatre, screen, and showtime, so lock contention for one blockbuster release does not affect unrelated shows.

6.2 Maintainability

- Bounded contexts — each module owns one concern (catalog, inventory, reservation, payment, notification), so a change to payment logic cannot silently break seat-locking behavior.
- Clear internal contracts — modules communicate only through defined REST calls or published events, never through shared mutable state, which keeps modules independently testable with JUnit/Mockito.
- Adapter isolation — all third-party integrations (payment gateway, SMS/email/push, partner feeds) sit behind the Integration Module, so a provider swap (e.g., Stripe to Razorpay) touches one module only.
- Incremental migration path — the modular-monolith structure can be split into independently deployed microservices module-by-module as team size or load grows, without a rewrite.

7. Summary

This architecture translates the SRD requirements — particularly the concurrency and locking strategy — into a concrete, layered system: a stateless service layer for business logic, Redis/Redisson for correctness under concurrent seat contention, PostgreSQL as the durable system of record, and clearly bounded modules for catalog, inventory, reservation, waitlist, payment, and notification concerns.

The design supports the core DSA-driven features (2D seat grid, HashMap-based $O(1)$ status lookups, PriorityQueue-based VIP waitlisting, and timer-driven hold expiry) while remaining scalable and maintainable as CineSeat grows from a single-theatre prototype toward a multi-theatre, multi-city platform.